```
    it "makes the score 0-15 if the receiver wins a point" do
      ts.give_point_to(:receiver)
      expect(ts.score).to eq("0-15")
    end

    it "makes the score 15-15 after they both win a point" do
      ts.give_point_to(:receiver)
      ts.give_point_to(:server)
      expect(ts.score).to eq("15-15")
    end
  end
end
```

The let block is only evaluated when the associated variable is used, and then the block is evaluated once, and further uses of that variable use the stored value from the first evaluation.

We're going to stop here, but I suggest that you might want to take this code and continue to develop it. Write expectations such as these:

```
it "is 40-0 after the server wins three points"
it "is W-L after the server wins four points"
it "is L-W after the receiver wins four points"
it "is Deuce after each wins three points"
it "is Advantage-server after each wins three points and the server gets one more"
```

RSpec and Matchers

In the previous code, we kind of ran right past RSpec's matchers—lines like expect(@ts.score).to eq("15-0"). RSpec has a rich syntax of matchers to cover the same ground that minitest does with different assertions.

We've already seen eq, but that matcher is a little unusual. Many of RSpec's matchers start with be, as with this set of matchers that cover basic logic:

```
expect(value).to be_truthy
expect(value).to be_falsey
expect(value).to be_an_instance_of(Product)
expect(value.price).to be > 10
expect(value.price).to be_between(5, 15)
```

You can substitute any Ruby comparison operator instead of the greater than symbol in be >.

RSpec also provides matchers for structured data, like objects, arrays, hashes, and strings:

```
expect(array).to contain_exactly(:a, :b, :c)

expect(hash).to include(key: value)

expect(string).to start_with("abc")
expect(string).to end_with("xyz")

expect(instance).to have_attriubtes(color: "blue")

expect(array_string_or_hash).to include("value")
```